

Activation-Confirmed Dual-Channel HIL Fault Injection for Payload-Link Supervision

Abstract—Payload communication is an important fault boundary in embedded controllers: a payload can become silent, return a corrupted response, or answer after a deadline. This paper presents an activation-confirmed hardware-in-the-loop (HIL) method that exercises those behaviors on a physical UART link. A host orchestrates a controller board and a commandable payload-simulator board through independent serial-observation channels. A trial is accepted only after payload-side confirmation of both fault activation and NORMAL restoration; controller-side markers then define detection and recovery. The campaign comprises two 65-s nominal windows and 90 sequential trials on one board pair, with 30 trials per fault mode. All 90 trials were valid, and the predefined detector and recovery markers were each observed in 30/30 valid trials for every mode. Host-observed command-to-marker medians ranged from 359.5 to 406.0 ms for detection and from 296.5 to 359.0 ms for recovery. The contribution is a dual-channel protocol that distinguishes requested, activated, detected, restored, and recovered events, supported by a hash-locked empirical record. Results are bounded to the evaluated setup and are not a device-population reliability or controller-internal timing estimate.

Index Terms—hardware-in-the-loop, fault injection, embedded systems, payload link, reproducibility

I. INTRODUCTION

Embedded controllers commonly supervise peripheral or payload communication by checking whether responses arrive, whether their integrity fields are valid, and whether a previously unavailable peer returns to service. Testing only a healthy peer leaves those paths weakly observed. Conversely, an injection campaign can become difficult to interpret when the experiment records only a sent command, mixes controller and injector evidence, or reports host timestamps as if they were internal execution times.

Fault injection has long been used to exercise fault-handling mechanisms under controlled conditions. Arlat *et al.* characterize an experiment through faults, activations, readouts, and measures (the FARM sets) [1]. HIL studies place real controller hardware against a controllable environment and inject abnormal signals without modifying the system under test [2], [3]. Nanosatellite research has also injected faults into a payload communication bus and reused modeled interoperability tests with physical subsystems [4], [5]. These principles motivate a focused experiment at a payload communication endpoint.

This paper evaluates one physical controller/payload-simulator pair. The simulator exposes three commandable behaviors: silence, CRC corruption, and response delay. A host harness confirms the selected behavior on the simulator channel before accepting a controller-side detection event.

After a fixed observation interval, the harness restores normal behavior, confirms that restoration, and observes a predefined recovery marker. Every raw serial record retains a host timestamp, exact bytes, and readable rendering. The resulting claim scope is locked before manuscript assembly: quantitative statements are projected from hash-verified frozen summaries, and unsupported extrapolations are excluded.

The paper makes three bounded contributions:

- an activation-confirmed dual-channel HIL procedure that separates requested, activated, detected, restored, and recovered events;
- a descriptive evaluation of nominal behavior, predefined detection/recovery outcomes, and host-observed command-to-marker latency for three payload-link behaviors; and
- an evidence-locking workflow that connects manuscript tables to immutable experiment summaries and validation artifacts.

The study does not compare against a C0 ablation, treat sequential trials as independent hardware replicas, or infer controller-internal timing. These boundaries are central to the interpretation rather than post-hoc qualifications.

II. RELATED WORK

The FARM formulation separates the injected fault, its activation, system readouts, and resulting measures [1]. This separation is important when a command to an injector may not prove that the requested behavior became active. Experimental conclusions also depend on the chosen fault model and observation boundary [6]. In this study, the commanded payload behavior is the fault, payload-side mode confirmation is activation evidence, controller markers are readouts, and observed counts plus host-timestamp distributions are measures.

Recent automotive HIL frameworks combine a real controller with a simulated environment and programmatic signal faults. Abboush *et al.* first demonstrated real-time sensor/actuator fault injection around an ECU prototype [2], then extended the approach into a virtual testing framework for single and simultaneous faults in varied driving scenarios [3]. Those studies emphasize real-time dynamic-system validation. The present work instead targets a framed serial endpoint and the evidentiary sequence from a fault request through confirmed activation and restoration to controller detection and recovery.

Closest to the application boundary here, Batista *et al.* place a failure-emulator mechanism in the NanosatC-BR2 payload I²C channel and exercise robustness tests in MIL and HIL

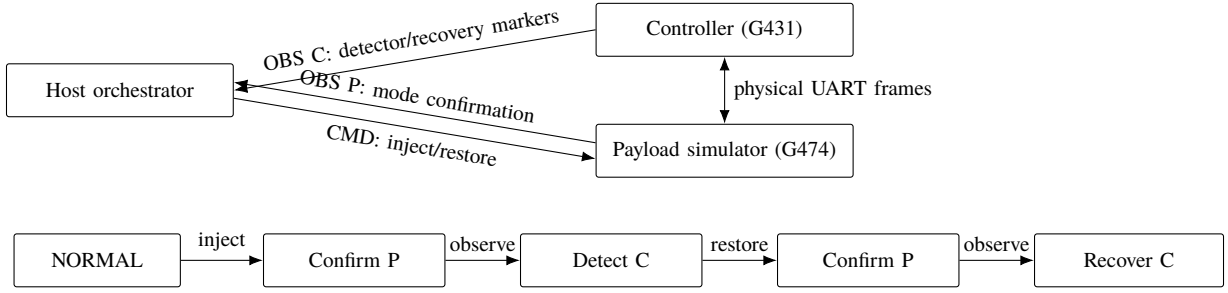


Fig. 1. Dual-channel setup and accepted trial sequence. OBS C and OBS P are independent host serial channels; payload confirmation gates scoring after injection and restoration.

TABLE I
SELECTED ADJACENT HIL EVALUATION BOUNDARIES.

Study	Boundary	Evaluation emphasis
Abboush [2], [3]	Automotive model signals	Dynamic behavior under programmatic faults
Batista [4]	Nanosatellite I ² C bus	Interoperability and robustness requirements
Conceição [5]	Staged CubeSat MIL/HIL	Model-based interoperability tests
This work	UART payload endpoint	Confirmation-gated dual-channel observations

configurations [4]. Conceição and Mattiello-Francisco later systematize CubeSat interoperability models, generated test cases, and staged reuse from simulated boards to physical subsystems [5]. These works establish communication-bus fault injection and systematic interoperability testing; they do not report the specific empirical construct evaluated here: independent host observation of controller and payload endpoints, acceptance gated by explicit activation/restoration confirmations, and byte-preserving per-trial traces. Evidence locking supports reproducibility of that construct but is not claimed as its sole novelty.

III. SYSTEM AND RESEARCH QUESTIONS

A. Payload-Link Supervision

The setup consists of a NUCLEO-G431RB controller and a NUCLEO-G474RE payload simulator connected by a UART at 115200 8N1 with common ground. The controller periodically exchanges framed messages with the payload. Its observable supervision paths include response timeout, CRC rejection, transition to an offline state, and a later recovery marker. Boot and runtime mechanisms provide architectural context, but the empirical contribution is limited to the payload link.

The payload simulator accepts host commands for NORMAL, SILENT, BAD_CRC, and DELAYED modes. SILENT withholds a response. BAD_CRC returns a response with an invalid CRC. DELAYED waits 250 ms before responding, exceeding the inspected 100-ms controller deadline. A second host serial channel captures the simulator’s exact mode confirmation. Thus, a sent command alone is insufficient activation evidence.

B. Locked Research Questions

The study asks only the following descriptive questions:

RQ1: What payload-link status and counter behavior is observed during the predefined pre- and post-campaign nominal (N0) windows?

RQ2: For SILENT, BAD_CRC, and DELAYED, in how many valid sequential trials are activation, the mode-specific detector, NORMAL restoration, and recovery markers observed?

RQ3: What are the per-mode distributions of host-observed command-to-detector-marker and restore-command-to-recovery-marker latency?

N0 denotes a healthy nominal condition; it is not a comparative or ablation baseline.

IV. EVALUATION METHOD

A. Nominal Windows

A 65-s N0 window was captured before the campaign and another after it. A window was valid if it contained at least 10 controller status records, all records reported ONLINE, the successful-message counter increased strictly, timeout/CRC/sequence/recovery counter deltas were zero, and no prohibited transition, rejection, recovery, or post-start link-start marker appeared.

B. Trial Protocol and Outcomes

Seed 20260830 fixed the order of 90 sequential trials and their pre-injection offsets: 30 trials for each mode. Every trial used the same physical pair. As Fig. 1 shows, the protocol first commanded NORMAL and observed the precondition, waited the seeded offset, sent the selected mode, and required the matching simulator confirmation. The controller was then observed for 4 s. The harness sent NORMAL, required the simulator’s NORMAL confirmation, and observed the controller for a further 3 s.

For SILENT, the predefined detector was the first post-injection timeout or offline marker. For BAD_CRC, it was the first explicit CRC-rejection marker. For DELAYED, it was the first timeout or offline marker, consistent with the configured delay and inspected deadline. OFFLINE was retained as a secondary observation and was not required for BAD_CRC detection. Recovery required a controller recovery marker after

the restore command and a confirmed NORMAL restoration. A restart field recorded only the presence or absence of a post-injection link-start marker; its absence was not interpreted as independent reset evidence.

A trial was valid only when its invalid-reason field was empty and both activation and restoration were confirmed on the simulator channel. Machine validation cross-checked 90 plan rows, 90 result rows, and all 180 controller/payload raw logs and reported no issues.

C. Measurement and Analysis

Detection latency is the host timestamp of the first pre-defined controller detector marker minus the host timestamp at injection-command send. Recovery latency is the host timestamp of the controller recovery marker minus the host timestamp at NORMAL-command send. These intervals include serial transport, USB, driver buffering, host scheduling, and timestamping effects; there was no host/MCU clock synchronization.

Analysis is descriptive. Per mode, we report detection and recovery counts and observed proportions. Latency summaries contain sample count, median, inclusive-linear quartiles, interquartile range (IQR), minimum, and maximum. No confidence interval, hypothesis test, or between-mode ranking is reported. The observations are sequential measurements on one setup, not independent device replicates; device-population uncertainty cannot be estimated from this single board pair.

D. Construct Validity and Controls

The intervention construct is a confirmed endpoint behavior, not merely a host command. The payload channel had to report the requested mode after injection and NORMAL after restoration; otherwise the trial was invalid. This supports temporal attribution within the protocol but does not make silence, CRC corruption, or delay equivalent to an electrical, environmental, or radiation-induced fault. The response construct is likewise operational: detection and recovery mean that the predefined controller marker appeared in the permitted observation phase. They do not expose every internal state transition.

Three controls reduce ambiguity without creating independent replication. First, each trial begins from an observed NORMAL precondition, and fixed fault/recovery windows separate detection from recovery scoring. Second, the seeded order and pre-injection offsets make the executed sequence reproducible while avoiding an unrecorded adaptive order. Third, controller and payload bytes are recorded separately before the result row is validated against both raw logs. These controls strengthen within-setup traceability. They do not remove shared hardware, host, thermal, or state-history conditions, and they cannot support population-level uncertainty estimates.

V. RESULTS

A. RQ1: Nominal Behavior

Both N0 windows were valid. Each lasted 65.0 s and contained 13 ONLINE status records. The successful-message

TABLE II
VALIDATED N0 WINDOWS. ONLY WITHIN-WINDOW DELTAS ARE INTERPRETED.

Phase	Status	ok first-last	TO	CRC	SEQ	REC
Pre	13	1835–1955	0	0	0	0
Post	13	3049–3169	0	0	0	0

TABLE III
OBSERVED OUTCOMES BY INJECTED MODE. PROPORTIONS ARE DESCRIPTIVE OBSERVATIONS ON ONE BOARD PAIR.

Mode	Valid	Detect	Recover	Offline	Start
SILENT	30/30	30/30	30/30	30/30	0/30
BAD_CRC	30/30	30/30	30/30	30/30	0/30
DELAYED	30/30	30/30	30/30	30/30	0/30

counter increased strictly from 1835 to 1955 before the campaign and from 3049 to 3169 after it. Within both windows, timeout, CRC, sequence, and recovery counter deltas were zero, and the prohibited-marker list was empty (Table II). The different absolute counter levels reflect preceding activity; only within-window changes are evaluated.

B. RQ2: Predefined Outcomes

All 90 planned trials were retained as valid. Each mode had 30 confirmed activations and 30 confirmed NORMAL restorations. The predefined detector and recovery markers were each observed in 30/30 valid trials per mode (observed proportion 1.0). OFFLINE was observed in 30/30 trials for each mode during the configured fault window, including BAD_CRC, where it was not the detection criterion. No post-injection link-start marker was recorded (Table III); this marker absence is not proof that no reset occurred.

The 30/30 values describe the retained sequential observations rather than draws from an identified device population. Their evidentiary role is to show that no valid trial in this campaign lacked the predefined marker. They are not converted into a reliability estimate or a confidence bound because repeated runs on the same hardware do not establish the sampling model required for such a population claim.

C. RQ3: Host-Observed Latency

Table IV reports both host-observed intervals. Command-to-detector-marker medians were 383.0 ms for SILENT, 359.5 ms for BAD_CRC, and 406.0 ms for DELAYED. Restore-command-to-recovery-marker medians were 297.0, 359.0, and 296.5 ms, respectively. The IQRs and full observed ranges show substantial within-mode dispersion. These summaries are not compared inferentially and do not define internal controller execution bounds.

VI. DISCUSSION AND LIMITATIONS

The dual-channel protocol closes a specific evidentiary gap between requesting a behavior and observing that the payload simulator entered it. Within the frozen setup, all retained injections produced their predefined controller detector evidence and were followed by a recovery marker

TABLE IV
HOST-OBSERVED COMMAND-TO-MARKER LATENCY (MS), $n = 30$ PER ROW.
QUANTILES USE THE FROZEN INCLUSIVE-LINEAR DEFINITION.

Mode	Int.	Median	Q1	Q3	IQR	Min-Max
SILENT	Det.	383.0	288.75	485.0	196.25	125–578
BAD_CRC	Det.	359.5	171.25	464.25	293.0	78–531
DELAYED	Det.	406.0	297.0	512.0	215.0	156–610
SILENT	Rec.	297.0	144.25	375.0	230.75	78–516
BAD_CRC	Rec.	359.0	113.75	417.25	303.5	47–531
DELAYED	Rec.	296.5	187.25	425.5	238.25	94–531

after confirmed restoration. The N0 windows provide bounded healthy observations on both sides of the campaign. Together, these records support a repeatable procedure for inspecting payload-link supervision on this pair. They do not establish behavior on an unobserved device population, and population-level uncertainty cannot be estimated from one physical pair.

The experiment has several deliberate limitations. First, all trials used one physical board pair and occurred sequentially. Trial outcomes may therefore share device, thermal, state-history, host, and timing conditions. Second, nominal evidence is limited to two 65-s windows; it does not establish long-duration stability. Third, no fair C0 ablation was constructed, so neither a treatment effect nor comparative superiority is supported. Fourth, timing is based on host command and serial-receipt timestamps. UART, USB, operating-system, and scheduling delays are included, and MCU-internal latency is unknown.

Fifth, outcomes are marker based. A recorded detector or recovery marker supports the corresponding protocol definition but not every internal state transition. Conversely, absence of a link-start marker does not independently prove the controller never reset. Literal heartbeat or watchdog status strings were not treated as independent health measurements. Sixth, only silence, bad CRC, and one delayed-response setting were exercised under fixed observation windows. Other delay magnitudes, corruption patterns, intermittent behavior, concurrent loads, and recovery horizons remain untested.

Finally, no environmental stress was applied: temperature, vibration, electromagnetic conditions, power transients, and radiation were not varied or measured. The evidence therefore provides no environmental robustness, flight-qualification, mission-assurance, or complete-system validation result. Binary-to-readback agreement strengthens provenance for the evaluated firmware, but it does not add trials or broaden the empirical population.

VII. REPRODUCIBILITY AND EVIDENCE LOCK

The campaign package retains a fixed seed and run plan, per-trial result rows, 180 raw serial logs, validation output, descriptive summaries, firmware, and SHA-256 inventories. The raw format records host UTC time, exact received bytes, and escaped text. A separate table generator first verifies the frozen hashes of the campaign summary, campaign validation, and both N0 validations, then deterministically projects the

manuscript CSV tables. The generator records its own hash and the input/output hashes.

This mechanism supports inspection of how every number in Tables II–IV reaches the manuscript. It also prevents an unnoticed change in a source summary from silently producing new tables. Re-execution on different hardware remains untested, and anonymous review omits any author-revealing repository location. Artifact access can be provided in a non-identifying form or after review according to venue policy.

VIII. CONCLUSION

This paper presented activation-confirmed dual-channel HIL fault injection for payload-link supervision using one controller board, one commandable payload-simulator board, and independent host observation channels. Two bounded nominal windows and 90 sequential injected trials provide a traceable description of predefined detection and recovery markers and host-observed command-to-marker latency distributions. The methodological result is setup bounded: confirmed link-endpoint behavior can be connected to controller evidence and immutable logs without converting repeated observations into claims about independent devices or internal timing. Future work may expand device replication, fault models, observation duration, and independent instrumentation; those extensions are not implied by the present dataset.

REFERENCES

- [1] J. Arlat, M. Aguera, L. Amat, Y. Crouzet, J.-C. Fabre, J.-C. Laprie, E. Martins, and D. Powell, “Fault injection for dependability validation: A methodology and some applications,” *IEEE Transactions on Software Engineering*, vol. 16, no. 2, pp. 166–182, 1990.
- [2] M. Abboush, D. Bamal, C. Knieke, and A. Rausch, “Hardware-in-the-loop-based real-time fault injection framework for dynamic behavior analysis of automotive software systems,” *Sensors*, vol. 22, no. 4, p. 1360, 2022.
- [3] M. Abboush, C. Knieke, and A. Rausch, “A virtual testing framework for real-time validation of automotive software systems based on hardware in the loop and fault injection,” *Sensors*, vol. 24, no. 12, p. 3733, 2024.
- [4] C. L. G. Batista, A. C. Weller, E. Martins, and M. de Fátima Mattiello-Francisco, “Towards increasing nanosatellite subsystem robustness,” *Acta Astronautica*, vol. 156, pp. 187–196, 2019.
- [5] C. A. P. L. Conceição and M. de Fátima Mattiello-Francisco, “Systematic testing approach for communicating software embedded in nanosatellites focusing on interoperability faults,” *Journal of Aerospace Technology and Management*, vol. 17, p. e1925, 2025.
- [6] T. Piper, S. Winter, N. Suri, and T. E. Fuhrman, “On the effective use of fault injection for the assessment of AUTOSAR safety mechanisms,” in *2015 11th European Dependable Computing Conference*, 2015, pp. 85–96.